

Arquitectura del sistema y decisiones de diseño

Por qué el sistema funciona como funciona

Este documento explica **por qué** el sistema está construido como está, no qué hace (eso está en la referencia de módulos).

El problema que resuelve

Las organizaciones reciben miles de comentarios en redes sociales durante una crisis. Revisarlos manualmente es lento y subjetivo. El sistema automatiza la detección temprana: clasifica cada comentario por sentimiento y alerta cuando el nivel de negatividad supera un umbral configurable, permitiendo al equipo priorizar su atención.

Por qué la entrada es un archivo CSV (y no una API en tiempo real)

El sistema recibe como entrada cualquier archivo CSV con la estructura `id, texto, fecha` (la columna `usuario` es opcional y se aprovecha cuando existe). Esta decisión es deliberada:

- **La API oficial de X eliminó su tier gratuito en 2023**, y su costo actual (desde USD 200 mensuales en el tier Basic) la hace inviable para un proyecto académico.
- **Las alternativas de scraping no autorizado fueron evaluadas y descartadas:** infringen los términos de servicio de la plataforma, pueden ocasionar la suspensión de las cuentas utilizadas y no garantizan disponibilidad estable.

A cambio, el contrato de entrada es un formato universal: el sistema procesa datos de cualquier fuente que exporte CSV con esas columnas, sin modificar el código — el dataset propio del proyecto (`comentarios_x.csv`), datasets públicos de Kaggle (adaptados con `convertir_kaggle.py`), o una exportación futura desde una API de pago, que queda planteada como trabajo futuro.

Para el desarrollo y la demostración se usa `comentarios_x.csv` : 69 comentarios en español que simulan publicaciones de X, construidos a partir de casos públicos documentados. El dataset incluye ruido deliberado (duplicados, símbolos, jerga) — no es una debilidad sino un dato de prueba diseñado: permite verificar el proceso de limpieza, deduplicación y filtrado en condiciones realistas.

Arquitectura de módulos

El sistema sigue un pipeline lineal en el que cada módulo tiene una responsabilidad única y pasa su salida al siguiente:



`main.py` orquesta el pipeline y expone todas las operaciones a través de un menú interactivo de consola. `convertir_kaggle.py` es una utilidad externa al pipeline: adapta el dataset de Kaggle al contrato de entrada.

Cómo funciona el análisis de sentimiento

El enfoque léxico

El sistema usa un **diccionario léxico ponderado** en lugar de modelos de machine learning. Cada palabra del léxico tiene un peso numérico: +3 para palabras muy positivas (ej: "maravilloso"), -3 para muy negativas (ej: "fraude"), e intermedios para gradaciones.

Por qué léxico en lugar de ML: el proyecto usa solo la biblioteca estándar de Python, sin dependencias externas ni modelos preentrenados de IA (Transformers/Hugging Face quedan explícitamente fuera del alcance). Un léxico en español con 256 entradas funciona bien para comentarios de redes sociales, donde las crisis se expresan con palabras de alto impacto emocional. Además, el enfoque léxico es completamente transparente: cada puntuación es auditable palabra por palabra.

Por qué el léxico incluye jerga peruana: el dataset del proyecto proviene de casos públicos en redes sociales peruanas, donde la

negatividad se expresa con coloquialismos e insultos locales que un léxico genérico del español no capturaría. Sin esas entradas, comentarios claramente hostiles puntuarían como neutros y el índice de crisis quedaría subestimado.

Por qué hay una superclase (AnalizadorBase)

`AnalizadorSentimiento` hereda de `AnalizadorBase`, que define el contrato común de los analizadores (`describir()` y un `clasificar()` genérico que las subclases sobrescriben). Además de ser la evidencia de herencia y polimorfismo de la Unidad 2, este diseño permite agregar en el futuro otros analizadores (por ejemplo, uno por reglas de emojis) sin tocar el resto del pipeline: cualquier objeto que respete el contrato de `clasificar()` encaja en `analizar_lote()`.

Intensificadores

Palabras como "muy" (+0.5), "absolutamente" (+1.0) o "totalmente" (+0.5) suman su propio peso a la puntuación bruta. El efecto es que "absolutamente terrible" acumula más carga que "terrible" solo. Es un modelado simple (suma en vez de multiplicación del peso vecino), pero suficiente para que la intensidad del lenguaje influya en la categoría final sin complicar el algoritmo.

Normalización por longitud de texto

```
puntuacion_normalizada = (puntuacion_bruta / n_palabras) * 10
```

Por qué normalizar: un comentario de 100 palabras con tres insultos acumula -9 de puntuación bruta, pero uno de 2 palabras con un solo insulto acumula -3. Sin normalizar, el comentario largo parecería peor. La normalización los trata por densidad negativa, no por cantidad bruta.

Consecuencia observable: textos muy cortos producen puntuaciones extremas. Un insulto de 1 palabra con peso -3 puntúa -30.0, mientras el mismo insulto en una frase de 3 palabras puntúa cerca de -10.0. Esto es matemáticamente correcto pero puede sorprender al leer el reporte. La categorización por umbrales (< -2.0 = muy negativo) mitiga el efecto.

Manejo de negadores

Palabras como "no", "nunca", "jamás", "ni", "tampoco" y "sin" funcionan como negadores: invierten el peso de la palabra que les sigue, y la pareja consume ambos tokens.

```
"no bueno"    → bueno tiene peso +2, pero negado queda -2
"nunca malo"  → malo tiene peso -2, negado queda +2
```

Cuando un negador aparece al final de una oración (sin siguiente palabra), no suma ningún peso — su papel es solo gramatical.

Por qué la deduplicación es por texto limpio

Dos comentarios son duplicados si producen el mismo texto al aplicar `limpiar_texto()`, sin importar su id o fecha.

Motivación: en X los usuarios copian y pegan el mismo mensaje múltiples veces (brigadas de comentarios coordinados). Contar cada copia por separado inflaría artificialmente el porcentaje de negativos y podría disparar una alerta global falsa. La deduplicación por contenido limpio filtra la amplificación artificial.

Consecuencia: si dos usuarios distintos publican exactamente el mismo texto (ej: un hashtag de campaña), solo el primero se analiza. En la práctica, los comentarios coordinados son una señal de crisis en sí mismos, y la alerta global se dispara igualmente con el índice de negativos sobre el conjunto de comentarios únicos.

Por qué hay un filtro de ruido

Tras deduplicar, la ingesta descarta los registros cuyo texto limpio tiene menos de 3 caracteres (`LONGITUD_MINIMA`): símbolos sueltos ("."), restos de enlaces o menciones vacías. Estos registros no son comentarios reales; dejarlos pasar inflaría el denominador del índice de crisis con falsos "neutros" y diluiría el porcentaje de negativos. En el dataset del proyecto este filtro descarta 2 registros.

Por qué la columna usuario es opcional pero se aprovecha

El contrato mínimo de entrada exige solo `id`, `texto`, `fecha` para que cualquier dataset sirva (el de Kaggle, por ejemplo, no trae autor). Pero cuando la fuente sí incluye `usuario`, el sistema lo propaga por todo el pipeline: aparece en las alertas de consola, se persiste en ambas tablas de SQLite y en el CSV de negativos, y alimenta la sección "usuarios con más comentarios negativos" del reporte — útil para identificar cuentas que concentran la hostilidad en una crisis coordinada.

Diseño de las alertas

El sistema tiene dos tipos de alerta independientes:

- **Alerta individual:** se dispara cuando un comentario específico tiene `puntuacion < umbral` (por defecto -2.0). Detecta un comentario particularmente agresivo o crítico.
- **Alerta de crisis global:** se dispara cuando el porcentaje total de comentarios negativos + muy negativos supera el `porcentaje_crisis` (por defecto 30%). Detecta que la negatividad es sistémica, no un caso aislado.

Los dos mecanismos son complementarios: es posible tener muchas alertas individuales sin alerta global (si el total de comentarios es grande), y también es posible tener alerta global sin alertas individuales (si hay muchos comentarios levemente negativos, ninguno muy grave).

El campo `activa` de la tabla `alertas` (1 = activa, 0 = resuelta) permite el ciclo de vida completo de una alerta con los cuatro verbos DML: se inserta al detectarse, se consulta en el monitoreo, se actualiza al resolverse y se elimina al depurar el historial.

Por qué cuatro formatos de persistencia

Formato	Uso principal
TXT	Revisión humana directa — legible sin herramientas
CSV filtrado	Importar negativos en Excel o herramientas BI
Pickle	Restaurar el estado completo en Python para análisis posterior
SQLite	Consultas históricas, resolución de alertas, filtrado por categoría

Cada formato tiene consumidores distintos. El CSV es para el analista de negocio; el Pickle es para el desarrollador; SQLite es para el sistema operativo con historial; el TXT es para el reporte imprimible.

Estructura del estado de sesión

```
estado = {
  "datos_analizados": None,    # list[dict] | None
  "alertas":           [],     # list[dict]
  "estadisticas":      {},     # dict
  "umbral":            -2.0,   # float (configurable en opción 8)
}
```

Este diseño evita variables globales sueltas y hace que cada función de menú sea explícita sobre qué datos necesita y cuáles modifica. El umbral persiste en el estado durante la sesión; si el usuario lo cambia (opción 8) y luego carga un nuevo CSV (opción 1), el nuevo umbral se aplica.

Manejo de errores por capa

Capa	Qué captura	Cómo lo expone
<code>ingesta.py</code>	<code>FileNotFoundError</code> , <code>ValueError</code> (columnas), <code>UnicodeDecodeError</code> (fallback automático a latin-1)	Propaga al llamador
<code>persistencia.py</code>	<code>sqlite3.Error</code>	Envuelve en <code>RuntimeError</code>
<code>main.py</code> handlers	<code>FileNotFoundError</code> , <code>ValueError</code> , <code>RuntimeError</code> , <code>Exception</code>	Imprime mensaje de error en consola; no aborta el programa
<code>main.py</code> bucle principal	<code>KeyboardInterrupt</code> , <code>EOFError</code> , <code>Exception</code>	Ofrece confirmación de salida; continúa si se rechaza

El principio: los módulos internos propagan errores tipados; el menú los muestra en lenguaje natural y mantiene el sistema en ejecución.

Véase también

- Referencia de módulos** — API completa de cada función
- Tutorial de inicio** — ver el sistema en funcionamiento
- Guías de tareas comunes** — operar el sistema en la práctica